

NSR/AS Lab 3 – VPNs, Pattarapol Tantechasa, 103883220

Pattarapol Tantechasa
Student
Swinburne University of Technology
Melbourne, VIC, Australia
103883220@student.swin.edu.au

Abstract— This report introduces VPNs, with both theoretical background and hands-on configuration. The first part outlined theory of VPNs, their characteristics, capabilities and how it can be used to implement organizational security policy. Furthermore, practical lab activities to dive deeper into how to set up a simple encrypted VPN between the two Ubuntu machines using Linux VPN called OpenVPN, observe and analyze traffic behavior using Wireshark. Results confirmed that data is encrypted on the ethernet interface and decrypted within the tunnel. The lab reinforces key networking principles, demonstrates how VPN works and builds foundational knowledge in secure network communication.

Keywords—VPN, Tunnelling, OpenVPN, IP Address, Telnet, Encryption, Network Security.

I. INTRODUCTION TO VPNs

A Virtual Private Network (VPN) allows users to establish a protected network connection over public networks [1]. It enables secure transmission of sensitive data across untrusted networks such as the internet [2]. VPNs connect two devices through an encrypted tunnel, securing packet communication and masking IP addresses from external observers.

A. VPNs Characteristics and Capabilities

1) *Tunnelling*: VPNs rely on tunnelling protocols, this encapsulate data inside a secure communication channel. This allows different network layers to be used depending on the protocol. For example, IP Security (IPSec) uses Network Layer protocol (Layer 3) [3].

2) *Encryption*: Another core characteristics of a VPN, it ensures confidentiality even when data is transmitted across insecure public networks like internet [3].

3) *Data Integrity*: VPNs ensure that data transmitted through tunnel can't be altered by hash-based algorithms such as HMAC. Integrity is handled by The Authentication Header (AH) for authentication and The Encapsulating Security Payload (ESP) to provide both encryption and integrity [2], [4].

4) *Authentication*: VPNs authenticate users before establishing a connection, using method like CHAP or EAP [3], [5].

5) *Remote Access Connectivity*: VPNs allow mobile users and remote workers to securely access internal organizational resources as if they were on-site [3].

6) *Cost Effective*: Instead of using expensive leased lines from Internet Service Provider (ISP), organizations can use VPNs over public internet infrastructure to interconnect branch offices [3].

7) *Controlled Third-Party Access*: VPNs enable secure, selective access for partners, clients or contractors to organization sensitive information like stock on hand [3].

8) *Scalability and Flexibility*: VPNs support a large number of users and can scale effectively across various sizes of organisation from small to enterprises without significant infrastructure changes. Also, support multiple VPN types, which makes it flexible for different business needs [3].

B. How VPNs can be used to implement organisational security policy

VPNs play a vital role during implementation of security policy for an organization, as they enable secure remote access, maintaining data confidentiality and integrity, and controlling user access to sensitive resources [3]. Organization can ensure that only authorized users can access internal systems through authentication mechanisms such as CHAP, thereby enforcing access control policies [5]. Encryption protocols like IPSec are used to protect data in transit and help to hide source IP addresses by encapsulating traffic within secure tunnels, aligning with organizational requirements for confidentiality and integrity [4]. As well as, VPNs enforce authorization policies, allow control over which users or partners can access certain resources, from which locations and at what time [3]. As a result, VPNs are not only a tool for connectivity but also a critical enabler for applying and maintaining organizational cybersecurity policies [3].

II. OPENVPN BEHAVIOUR

This lab conducted a simple encrypted VPN between the two Ubuntu machines using Linux VPN software called OpenVPN. First, I began with setting up two Ubuntu machines and check IP Address of both machine with “*ifconfig*” command, as shown in Table I. As well as check connection between the two machines with “*ping*” command.

TABLE I. INITIAL VIRTUAL MACHINES IP ADDRESS

Virtual Machine	IP Address
Ubuntu 1	192.168.120.128
Ubuntu 2	192.168.120.129

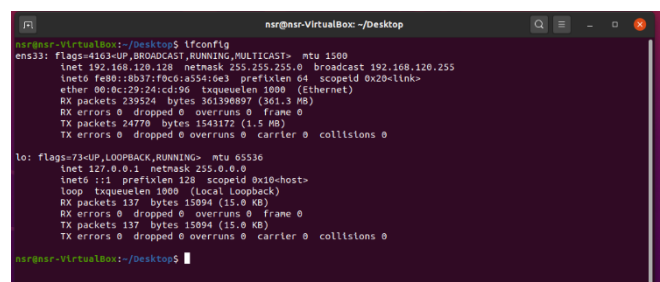


Fig. 1. Checking the IP Address of Ubuntu 1 machine.

```
nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.120.129 netmask 255.255.255.0 broadcast 192.168.120.255
    inet6 fe80::f424:ef02:f628:4a57 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:31:19:d3 txqueuelen 1000 (Ethernet)
    RX packets 259555 bytes 39126468 (391.2 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15233 bytes 975454 (975.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 126 bytes 14842 (14.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 126 bytes 14842 (14.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 2. Checking the IP Address of Ubuntu 2 machine.

```
nsr@nsr-VirtualBox:~/Desktop$ ping 192.168.120.129
PING 192.168.120.129 (192.168.120.129) 56(84) bytes of data.
 64 bytes from 192.168.120.129: icmp_seq=1 ttl=64 time=0.552 ms
 64 bytes from 192.168.120.129: icmp_seq=2 ttl=64 time=1.08 ms
 64 bytes from 192.168.120.129: icmp_seq=3 ttl=64 time=0.538 ms
 64 bytes from 192.168.120.129: icmp_seq=4 ttl=64 time=0.413 ms
 64 bytes from 192.168.120.129: icmp_seq=5 ttl=64 time=0.723 ms
 64 bytes from 192.168.120.129: icmp_seq=6 ttl=64 time=0.738 ms
 64 bytes from 192.168.120.129: icmp_seq=7 ttl=64 time=0.807 ms
 64 bytes from 192.168.120.129: icmp_seq=8 ttl=64 time=0.770 ms
 64 bytes from 192.168.120.129: icmp_seq=9 ttl=64 time=0.740 ms
 64 bytes from 192.168.120.129: icmp_seq=10 ttl=64 time=0.838 ms
--
192.168.120.129 ping statistics --
 10 packets transmitted, 10 received, 0% packet loss, time 9180ms
 rtt min/avg/max/mdev = 0.413/0.721/1.082/0.176 ms
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 3. Ping to check connection between the two machines.

After connectivity is established, I generated the shared password named *mykey* on Ubuntu 1 machine with “*openvpn --genkey --secret mykey*” command. Verify that key has been created with “*ls -l*” command and examine the file with “*cat mykey*” command.

```
nsr@nsr-VirtualBox:~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ openvpn --genkey --secret mykey
nsr@nsr-VirtualBox:~/Desktop$ ls -l
total 4
-rw-r----- 1 nsr nsr 636 Apr  8 19:44 mykey
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 4. Generate and verify “mykey” on Ubuntu 1 machine.

```
nsr@nsr-VirtualBox:~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ cat mykey
#
# 2048 bit OpenVPN static key
#
-----BEGIN OpenVPN Static key V1-----
f37a36a8911997c80908b29f8e0a4db
804210043ec5d09b04ee452292b04
02f631d437fe0150a8e42d1f0de47
1d29282cf0088bdc0e0d03ff64d72a5
e38d0895d0b98ef4561cc101253e24a
19339993b0af7a2eb0dc0e140bd0b
2e3763efa7e2c0ffe33628f1ba1ffc0
27b339767ce3b03de83b45f0b5590f7
cf072e4f70e1f33e0b125e04d10f
ab3fc7be0fddc5c0b32f26f391225b
1b71e24fde9f9c11da717ca1c24c0e9
3c6cbf89f73a3022346c939b0dc053
1e793d8e0d0e1030ff7760937a0807
9c3a51442939445498307850a8f3f067
abd764dfc6f99abb5646117dc0d734
d4e4e5bcbca28b0a0b765131cc24
-----END OpenVPN Static key V1-----
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 5. Examine “mykey” on Ubuntu 1 machine.

Now that the shared key has been successfully generated, I copy and transfer *mykey* to Ubuntu 2 machine with Secure Copy (*scp*). Firstly, I installed and updated *ssh* (includes *scp*) on both machines with following command: “*sudo apt update*”, “*sudo apt-get install openssh-server*”.

```
nsr@nsr-VirtualBox:~/Desktop
Unpacking openssh-client (1:8.2p1-4ubuntu0.12) over (1:8.2p1-4ubuntu0.1) ...
Selecting previously unselected package ncurses-term.
Preparing to unpack .../ncurses-term_6.2-0ubuntu2.1_all.deb ...
Unpacking ncurses-term (6.2-0ubuntu2.1) ...
Selecting previously unselected package openssh-sftp-server.
Preparing to unpack .../openssh-sftp-server_1:8.2p1-4ubuntu0.12_and04.deb ...
Unpacking openssh-sftp-server (1:8.2p1-4ubuntu0.12) ...
Selecting previously unselected package openssh-server.
Preparing to unpack .../openssh-server_1:8.2p1-4ubuntu0.12_and04.deb ...
Unpacking openssh-server (1:8.2p1-4ubuntu0.12) ...
Selecting previously unselected package ssh-import-id.
Preparing to unpack .../ssh-import-id_5.10-0ubuntu1_all.deb ...
Unpacking ssh-import-id (5.10-0ubuntu1) ...
Setting up openssh-client (1:8.2p1-4ubuntu0.12) ...
Setting up ssh-import-id (5.10-0ubuntu1) ...
Attempting to convert /etc/ssh/ssh_import_id
Setting up ncurses-term (6.2-0ubuntu2.1) ...
Setting up openssh-sftp-server (1:8.2p1-4ubuntu0.12) ...
Setting up openssh-server (1:8.2p1-4ubuntu0.12) ...

Creating config file /etc/ssh/sshd_config with new version
Creating SSH2 RSA key; this may take some time ...
3072 SHA256:ozvXGTSS4HDK0L8JNs-hP8epZUvfrPbgJ8/3AWic root@nsr-VirtualBox (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:czpS1G1fC2D30a7u0xw2XJ0ERpdOpIoJ1PE24 root@nsr-VirtualBox (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:W0AfmfAIA7JvpxdH4uVWmpcJbN4888ilUfI+g root@nsr-VirtualBox (ED25519)
Created symlink /etc/systemd/system/ssh.service → /lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /lib/systemd/system/ssh.service.
rescue-ssh.target is a disabled or a static unit, not starting it.
Processing triggers for systemd (245.4-4ubuntu3.2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for ufw (0.36-6) ...
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 6. Installed and Updated *ssh* on Ubuntu 1.

```
nsr@nsr-VirtualBox:~/Desktop
Unpacking ncurses-term (6.2-0ubuntu2.1) ...
Selecting previously unselected package openssh-sftp-server.
Preparing to unpack .../openssh-sftp-server_1:8.2p1-4ubuntu0.12_and04.deb ...
Unpacking openssh-sftp-server (1:8.2p1-4ubuntu0.12) ...
Selecting previously unselected package openssh-server.
Preparing to unpack .../openssh-server_1:8.2p1-4ubuntu0.12_and04.deb ...
Unpacking openssh-server (1:8.2p1-4ubuntu0.12) ...
Selecting previously unselected package ssh-import-id.
Preparing to unpack .../ssh-import-id_5.10-0ubuntu1_all.deb ...
Unpacking ssh-import-id (5.10-0ubuntu1) ...
Setting up openssh-client (1:8.2p1-4ubuntu0.12) ...
Setting up ssh-import-id (5.10-0ubuntu1) ...
Attempting to convert /etc/ssh/ssh_import_id
Setting up ncurses-term (6.2-0ubuntu2.1) ...
Setting up openssh-sftp-server (1:8.2p1-4ubuntu0.12) ...
Setting up openssh-server (1:8.2p1-4ubuntu0.12) ...

Creating config file /etc/ssh/sshd_config with new version
Creating SSH2 RSA key; this may take some time ...
3072 SHA256:zmsJjBwmtGyXGcZmP5LSzKusku0VnuZ7b0pA root@nsr-VirtualBox (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:0WJE2L65DKpEaTumfvcvPYfDwIzEbxdsR+Mw0x0u root@nsr-VirtualBox (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:Ubu0d0yT4G+PwK1Z21V4b3LcpYg/PVZ0f0m0b root@nsr-VirtualBox (ED25519)
Created symlink /etc/systemd/system/ssh.service → /lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /lib/systemd/system/ssh.servic
e.
rescue-ssh.target is a disabled or a static unit, not starting it.
Processing triggers for systemd (245.4-4ubuntu3.2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for ufw (0.36-6) ...
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 7. Installed and Updated *ssh* on Ubuntu 2.

Now back to Ubuntu 1 machine, I run the command “*scp mykey nsr@192.168.120.129:Desktop/mykey*” to copy and transfer the shared key to Ubuntu 2 machine.

```
nsr@nsr-VirtualBox:~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ scp mykey nsr@192.168.120.129:Desktop/mykey
The authenticity of host '192.168.120.129 (192.168.120.129)' can't be established.
ECDSA key fingerprint is SHA256:0WJE2L65DKpEaTumfvcvPYfDwIzEbxdsR+Mw0x0u.
Are you sure you want to continue connecting (yes/no/[fingerprint]): y
Please type 'yes', 'no' or the fingerprint: y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.120.129' (ECDSA) to the list of known hosts.
nsr@192.168.120.129's password:
mykey
100% 636 682.5KB/s 00:00
nsr@nsr-VirtualBox:~/Desktop$
```

Fig. 8. Copy and Transfer *mykey* from Ubuntu 1 to Ubuntu 2 machine.

Ensuring *mykey* has been successfully transfer by run the command “*ls -l*” and “*cat mykey*” on Ubuntu 2 machine.

```

nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ ls -l
total 4
-rw-r----- 1 nsr nsr 636 Apr 8 19:53 mykey
nsr@nsr-VirtualBox:~/Desktop$ cat mykey
#
# 2048 bit OpenVPN static key
#
-----BEGIN OpenVPN Static key V1-----
f37a30a91197c9098b29f80ab4db
804210943ec5809bb4ee4522920e4
2a2f631d4377e0915e0042d1f30e47
1d29282cf608bdcdaec8d3ff04d72a5
e38d0895d0b9ef4561cc101253e24a
1935b99b5bafba2e0abdcac0140bad8
2e3762ef72c0eff320a018a1f60
27b339767ce3b03df83b45f0b5590f7
cf871e4f87a6e1fb3aeb0125ead4d16f
ab3fc7be0fddc5c0b31f326f391225b
1b71e24fde0f9c1d17fca1c4c0eb
1c6c6bf89f73a3022346c9e39b0bce53
1e793a84684be0103bfff709317a8007
9c3a51442939445408307850a8f5f067
6bd764dfcd99a0b5646d1fde0e434
8d4e645bcbca28eb0a8a7665131c24
-----END OpenVPN Static key V1-----
nsr@nsr-VirtualBox:~/Desktop$

```

Fig. 9. Verify successful transfer mykey on Ubuntu 2 machine.

TABLE II. VIRTUAL MACHINES IP ADDRESS AND TUNNEL END POINTS

Virtual Machine	IP Address	Tunnel End Points
Ubuntu 1	192.168.120.128	10.4.0.1
Ubuntu 2	192.168.120.129	10.4.0.2

Now I have begun the process of tunnel between the two machines, as shown in Table II. Setup tunnel on Ubuntu 1 machine by run the command “*sudo openvpn --remote 192.168.120.129 --dev tun1 --ifconfig 10.4.0.1 10.4.0.2 --verb 5 --secret mykey*”.

```

nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ sudo openvpn --remote 192.168.120.129 --dev tun1 --ifconfig 10.4.0.1 10.4.0.2 --verb 5 --secret mykey
Tue Apr 8 19:57:43 2025 us=817832 disabling NCP mode (--ncp-disable) because not in P2MP client or server mode
Tue Apr 8 19:57:43 2025 us=818028 Current Parameter Settings:
Tue Apr 8 19:57:43 2025 us=818074   config = '[UNDEF]'
Tue Apr 8 19:57:43 2025 us=818115   mode = 0
Tue Apr 8 19:57:43 2025 us=818156   persist_config = DISABLED
Tue Apr 8 19:57:43 2025 us=818195   persist_mode = 1
Tue Apr 8 19:57:43 2025 us=818235   show_ciphers = DISABLED
Tue Apr 8 19:57:43 2025 us=818275   show_digests = DISABLED
Tue Apr 8 19:57:43 2025 us=818314   show_engines = DISABLED
Tue Apr 8 19:57:43 2025 us=818354   genkey = DISABLED
Tue Apr 8 19:57:43 2025 us=818414   key_pass_file = '[UNDEF]'
Tue Apr 8 19:57:43 2025 us=818454   show_tls_ciphers = DISABLED
Tue Apr 8 19:57:43 2025 us=818495   connect_retry_max = 0
Tue Apr 8 19:57:43 2025 us=818535 Connection profiles [0]:
Tue Apr 8 19:57:43 2025 us=818574   proto = udp
Tue Apr 8 19:57:43 2025 us=818614   local = '[UNDEF]'
Tue Apr 8 19:57:43 2025 us=818654   local_port = '1194'
Tue Apr 8 19:57:43 2025 us=818693   remote = '192.168.120.129'
Tue Apr 8 19:57:43 2025 us=818733   remote_port = '1194'
Tue Apr 8 19:57:43 2025 us=818772   remote_float = DISABLED
Tue Apr 8 19:57:43 2025 us=818812   bind_defined = DISABLED
Tue Apr 8 19:57:43 2025 us=818852   bind_local = ENABLED
Tue Apr 8 19:57:43 2025 us=818891   bind_ipv6_only = DISABLED
Tue Apr 8 19:57:43 2025 us=818930   connect_retry_seconds = 5
Tue Apr 8 19:57:43 2025 us=818969   connect_timeout = 120
Tue Apr 8 19:57:43 2025 us=819009   socks_proxy_server = '[UNDEF]'
Tue Apr 8 19:57:43 2025 us=819048   socks_proxy_port = '[UNDEF]'
Tue Apr 8 19:57:43 2025 us=819149   tun_mtu_defined = ENABLED
Tue Apr 8 19:57:43 2025 us=819188   link_mtu = 1500
Tue Apr 8 19:57:43 2025 us=819227   link_mtu_defined = DISABLED

```

Fig. 10. Setup tunnel on Ubuntu 1 machine.

On Ubuntu 2, set up the tunnel in the reverse direction so that traffic can travel in both directions and verification.

```

nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ sudo openvpn --remote 192.168.120.128 --dev tun1 --ifconfig 10.4.0.2 10.4.0.1 --verb 5 --secret mykey
Tue Apr 8 20:01:37 2025 us=10748 disabling NCP mode (--ncp-disable) because not in P2MP client or server mode
Tue Apr 8 20:01:37 2025 us=11490 Current Parameter Settings:
Tue Apr 8 20:01:37 2025 us=11536   config = '[UNDEF]'
Tue Apr 8 20:01:37 2025 us=11570   mode = 0
Tue Apr 8 20:01:37 2025 us=11609   persist_config = DISABLED
Tue Apr 8 20:01:37 2025 us=11645   persist_mode = 1
Tue Apr 8 20:01:37 2025 us=11684   show_ciphers = DISABLED
Tue Apr 8 20:01:37 2025 us=11720   show_digests = DISABLED
Tue Apr 8 20:01:37 2025 us=11756   show_engines = DISABLED
Tue Apr 8 20:01:37 2025 us=11792   genkey = DISABLED
Tue Apr 8 20:01:37 2025 us=11828   key_pass_file = '[UNDEF]'
Tue Apr 8 20:01:37 2025 us=11863   show_tls_ciphers = DISABLED
Tue Apr 8 20:01:37 2025 us=11898   connect_retry_max = 0
Tue Apr 8 20:01:37 2025 us=11934   connection_profiles [0]:
Tue Apr 8 20:01:37 2025 us=11970   proto = udp
Tue Apr 8 20:01:37 2025 us=12006   local = '[UNDEF]'
Tue Apr 8 20:01:37 2025 us=12042   local_port = '1194'
Tue Apr 8 20:01:37 2025 us=12078   remote = '192.168.120.128'
Tue Apr 8 20:01:37 2025 us=12114   remote_port = '1194'
Tue Apr 8 20:01:37 2025 us=12150   remote_float = DISABLED
Tue Apr 8 20:01:37 2025 us=12186   bind_defined = DISABLED
Tue Apr 8 20:01:37 2025 us=12222   bind_local = ENABLED
Tue Apr 8 20:01:37 2025 us=12258   bind_ipv6_only = DISABLED
Tue Apr 8 20:01:37 2025 us=12294   connect_retry_seconds = 5
Tue Apr 8 20:01:37 2025 us=12330   connect_timeout = 120
Tue Apr 8 20:01:37 2025 us=12366   socks_proxy_server = '[UNDEF]'
Tue Apr 8 20:01:37 2025 us=12402   socks_proxy_port = '[UNDEF]'
Tue Apr 8 20:01:37 2025 us=12438   tun_mtu = 1500
Tue Apr 8 20:01:37 2025 us=12474   tun_mtu_defined = ENABLED

```

Fig. 11. Setup tunnel on Ubuntu 2 machine.

```

nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.120.129 netmask 255.255.255.0 broadcast 192.168.120.255
    ether 08:00:27:31:19:13 txqueuelen 1000 (Ethernet)
    RX packets 278801 bytes 419932563 (419.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 17604 bytes 1147929 (1.1 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 185 bytes 22511 (22.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 185 bytes 22511 (22.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tun1: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1500
    inet 10.4.0.2 netmask 255.255.255.255 destination 10.4.0.1
    inet6 fe80::754f:c2ae:84ca:9284 prefixlen 64 scopeid 0x20<link>
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 100 (UNSPEC)
    RX packets 1 bytes 48 (48.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5 bytes 240 (240.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

nsr@nsr-VirtualBox:~/Desktop$

```

Fig. 12. Verify successful tunnel on Ubuntu 2 machine.

After successfully set up tunnel between the two machines, I ping Ubuntu 1 from Ubuntu 2 using tunnel end point of Ubuntu 1, command as follows: “*ping 10.4.0.1*”. This way both machines communicate to each other through tunnel just set up.

```

nsr@nsr-VirtualBox: ~/Desktop
nsr@nsr-VirtualBox:~/Desktop$ ping 10.4.0.1
PING 10.4.0.1 (10.4.0.1) 56(84) bytes of data:
64 bytes from 10.4.0.1: icmp_seq=1 ttl=64 time=0.536 ms
64 bytes from 10.4.0.1: icmp_seq=2 ttl=64 time=1.50 ms
64 bytes from 10.4.0.1: icmp_seq=3 ttl=64 time=1.39 ms
64 bytes from 10.4.0.1: icmp_seq=4 ttl=64 time=0.905 ms
64 bytes from 10.4.0.1: icmp_seq=5 ttl=64 time=0.617 ms
64 bytes from 10.4.0.1: icmp_seq=6 ttl=64 time=0.710 ms
64 bytes from 10.4.0.1: icmp_seq=7 ttl=64 time=1.70 ms
64 bytes from 10.4.0.1: icmp_seq=8 ttl=64 time=0.716 ms
64 bytes from 10.4.0.1: icmp_seq=9 ttl=64 time=1.47 ms
64 bytes from 10.4.0.1: icmp_seq=10 ttl=64 time=1.54 ms
^C
... 10.4.0.1 ping statistics ...
10 packets transmitted, 10 received, 0% packet loss, time 9080ms
rtt min/avg/max/mdev = 0.536/1.107/1.699/0.426 ms
nsr@nsr-VirtualBox:~/Desktop$

```

Fig. 13. Ping Ubuntu 1 from Ubuntu 2 through tunnel.

Lastly, I observed traffic between the two-machine through tun1 and ens33 interfaces captured on Wireshark to answer questions below.

A. Is the traffic encrypted? Why?

When observing traffic on the tunnel interface (*tun1*) using Wireshark, it appears that the traffic is not encrypted. This is expected because the *tun1* interface represents the internal end of the VPN tunnel. We can see IP addresses of the tunnel endpoints and transport protocols. Encryption is meant to protect data in transit over the public network or from external observers. When we observe from the inside of the tunnel, the VPN software has already decrypted the packets, which is why the traffic is readable.

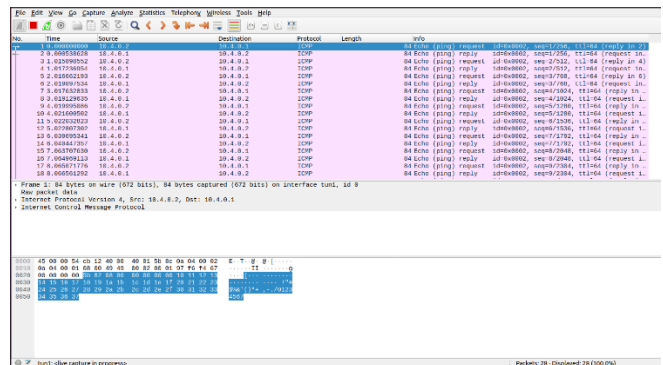


Fig. 14. Observing traffic on the tunnel interface using Wireshark.

B. Is the traffic encrypted when monitoring the ethernet interface?

Yes, the traffic is encrypted when observing the Ethernet interface (*ens33*). As we can see that Wireshark can't interpret the contents of the packets and flags them as "Malformed Packet" due to encryption.

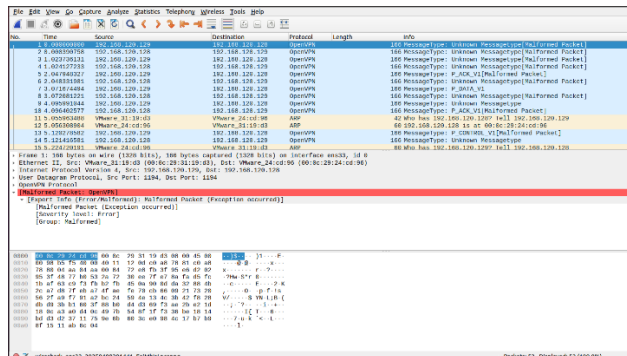


Fig. 15. Observing traffic on the ethernet interface using Wireshark.

C. How do you explain what you see?

We can see on the ethernet interface (*ens33*) is an encrypted VPN traffic inside an encapsulating protocol (*OpenVPN*). While the source and destination IP addresses of the VPN tunnel endpoints must remain visible for routing purposes, the payload itself is encrypted. Therefore, Wireshark shows OpenVPN packets using UDP port 1194 but cannot interpret their contents, which confirms that confidentiality is maintained.

D. On which interface is traffic encrypted and which interface not encrypted? Why?

I telnet from Ubuntu 2 to Ubuntu 1 with “*telnet 10.4.0.1*” command, then ensure successful logged into the remote machine with “*ifconfig*” command. Observing the tunnel interface (*tun1*) for traffic after running some simple command like “*ifconfig*” and “*ls*”.

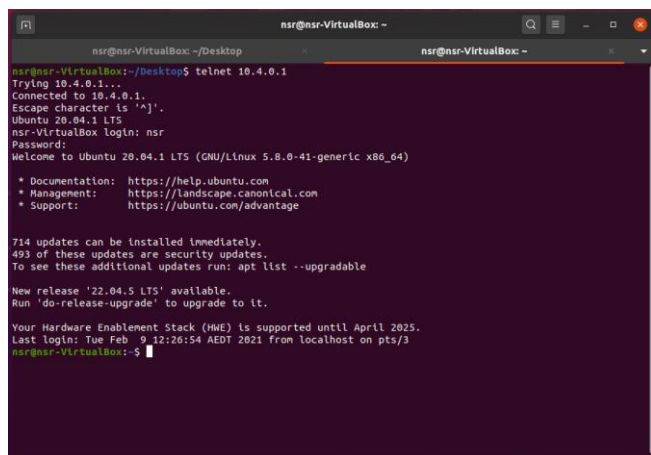


Fig. 16. Telnet from Ubuntu 2 to Ubuntu 1 through tunnel endpoints.

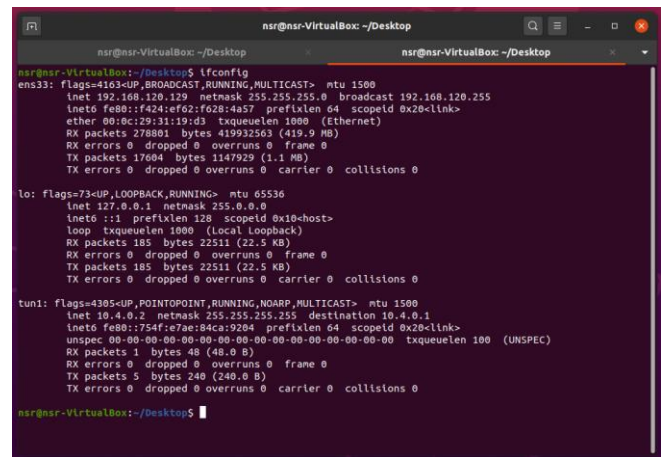


Fig. 17. Verify successfully logged into Ubuntu 1 through Telnet.

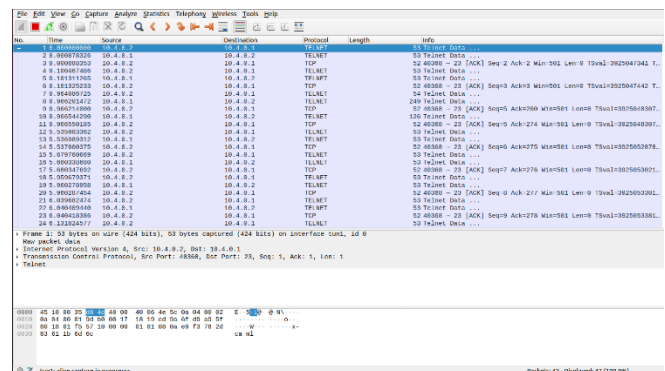


Fig. 18. Observing traffic on the tunnel interface (*tun1*).

Traffic on the ethernet interface (*ens33*) is encrypted because this interface handles the transmission of VPN packets over the network. These packets are encapsulated using OpenVPN over UDP and the original data, including Telnet sessions, is protected within the encrypted payload.

Unlike the tunnel interface (*tun1*), traffic is not encrypted because this interface represents the internal side of the VPN tunnel, where's data has already been decrypted by the VPN software, as shown in Figure 18.

III. CONCLUSION

In this lab, I successfully set up a simple encrypted VPN between the two Ubuntu machines using OpenVPN and verified its functionality through both command-line tools and packet inspection. Observing traffic using Wireshark on different interfaces, I was able to observe the behavior of encrypted and unencrypted traffic as it flows through and outside the VPN tunnel.

Traffic captured on the ethernet interface (*ens33*) appeared encrypted and unreadable, confirming that OpenVPN was securely encapsulating the data. Although, traffic captured on the tunnel interface (*tun1*) was visible and readable, such as ICMP (*ping*) and Telnet data.

In conclusion, I gained practical understanding of key VPN concepts, including tunneling, encapsulation and encryption. This hands-on experience deepened my understanding of how VPNs implement confidentiality, integrity and secure remote access, which are essential components of modern network security

REFERENCES

- [1] Kaspersky, "What is VPN? How It Works, Types of VPN", Accessed: April 17, 2025. [Online]. Available: <https://www.kaspersky.com/resource-center/definitions/what-is-a-vpn>
- [2] E. Barker, Q. Dang, S. Frankel, K. Scarfone, and P. Wouters, *Guide to IPsec VPNs*, USA: National Institute of Standards and Technology, 2020. [Online] Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-77r1.pdf>
- [3] P. Branch. (2025). Lecture_13_VPNs [PDF]. Available: https://swinburne.instructure.com/courses/66714/files/37592787?module_item_id=4969319
- [4] P. Branch. (2025). Lecture_14_VPNs [PDF]. Available: https://swinburne.instructure.com/courses/66714/files/37592785?module_item_id=4969320
- [5] P. Branch. (2025). Lecture_15_VPNs [PDF]. Available: https://swinburne.instructure.com/courses/66714/files/37592786?module_item_id=4969321